

Manual de uso ApachePoi + MySql



Grupo 8

Nome: Lucas Sousa Santos

Versão Arquivo:V1

Sumário

Classe Excel-	3
Classe RegistroAcidente	3
Método retornarVitimas:	4
Método retornarAcidente	5
Classe RetornarFks:	5
Método Fk_Concessionaria	6
Método Fk_rodovia:	6
Método Fk_ClasseAcidente:	7
Classe ExcelModeloAtual:	7
Método tratarExcel:	8
Informações importantes sobre o método:	8
Método analisarErros:	9
Classe ExcelModeloAntigo:	9
Método tratarExcel:	10
Informações importantes sobre método:	10
Método analisarErros:	11
Classe ConexaoBanco:	12
Classe ComandosJavaBanco:	13
Método save:	13
Método saveLog:	13
Método saveLote:	13
Classe Main:	15

Classe Excel-

A classe Excel, é uma classe abstrata que usaremos de superclasse e, teremos variável de instancia para usar de apoio no tratamento:

```
private String arquivoTratar; 3 usages
private Integer[] posicaoCelulasExcel; 3 usages
private Integer idPadrao; 3 usages
```

As três variáveis de instância ou dados bases como dito anteriormente são:

- arquivoTratar: Que vai passar o caminho do arquivo
- posicaoCelulasExcel: Passando exatamente os locais para ler
- idPadrao: para funcionar como um idBase no MySql e auxiliar nos updates

Além disso foi adicionado um método abstract “tratarExcel” obrigando as subclasses a implementa-los :

```
abstract List<RegistroAcidente> tratarExcel();
```

Classe RegistroAcidente

A classe RegistroAcidente, é uma classe onde vai salvar no java os dados de cada linha do arquivo no Excel deixando todos os dados salvos em uma variável específica.

Variáveis:

```
private Integer ID; 4 usages
private Integer nomeConcessionaria; 4 usages
private Integer rodovia; 4 usages
private String data; 3 usages
private Integer classeAcidente; 4 usages
private String tipoAcidente; 3 usages
private String meteor; 3 usages
private String VISIB; 3 usages
private String ILESA_INT; 3 usages
private String VIT_FATAL_INT; 3 usages
private String VIT_GRAVE_INT; 3 usages
private String VIT_LEVE_INT; 3 usages
private String VIT_MODERADA_INT; 3 usages
private String VIT_SEMINFO_INT; 3 usages
private String DENOMINACAO; 3 usages
private String MUNICIPIO; 3 usages
private String REGIONAL_DER; 3 usages
private String JURISDICA0; 3 usages
private String LAT_FINAL; 3 usages
private String LON_FINAL; 3 usages
```

No construtor tratamos os dados que vem como String e utilizamos da classe RetonarFKs que será explicada mais a frente no documento:

```
public RegistrarAcidente(Integer ID, String nomeConcessionaria, String rodovia, String data, String classeAcidente, String tipoAcidente, String natureza, String VISIB, String ILESA_INT, String VIT_FATAL_INT, String VIT_GRAVE_INT, String VIT_LEVE_INT, String VIT_MODERADA_INT, String VIT_SEMINFO_INT, String DENOMINACAO, String MUNICIPIO, String REGIONAL_OR, String JURISDICAO, String LAT_FINAL, String LON_FINAL) {
    RetonarFKs retorno = new RetonarFKs();
    this.ID = ID;
    this.nomeConcessionaria = retorno.FK_Concessionaria(nomeConcessionaria);
    this.rodovia = retorno.FK_rodovia(rodovia);
    this.data = data;
    this.classeAcidente = retorno.FK_classeAcidente(classeAcidente);
    this.tipoAcidente = tipoAcidente;
    this.natureza = natureza;
    this.VISIB = VISIB;
    this.ILESA_INT = ILESA_INT;
    this.VIT_FATAL_INT = VIT_FATAL_INT;
    this.VIT_GRAVE_INT = VIT_GRAVE_INT;
    this.VIT_LEVE_INT = VIT_LEVE_INT;
    this.VIT_MODERADA_INT = VIT_MODERADA_INT;
    this.VIT_SEMINFO_INT = VIT_SEMINFO_INT;
    this.DENOMINACAO = DENOMINACAO;
    this.MUNICIPIO = MUNICIPIO.replace(" ", "");
    this.REGIONAL_OR = REGIONAL_OR;
    this.JURISDICAO = JURISDICAO;
    this.LAT_FINAL = LAT_FINAL;
    this.LON_FINAL = LON_FINAL;
}
```

E temos os dois métodos para fazer a estrutura dos inserts o “retornarVitimas” e “retornarAcidente”.

Método retornarVitimas:

```
public String RetornarVitimas(){
    return String.format(
        "(%d, %s, %s, %s, %s, %s, %s)",
        ID,
        ILESA_INT,
        VIT_FATAL_INT,
        VIT_LEVE_INT,
        VIT_MODERADA_INT,
        VIT_GRAVE_INT,
        VIT_SEMINFO_INT
    );
}
```

Responsabilidade	Entrada	Saida
Usar as variáveis recebidas para criar um insert para a tabela Vitimas.	X	String com o insert

Método retornarAcidente

```
public String retornarAcidente(){ 1 usage  @ lucas-tdo *
    return String.format(
        "( %d,%d, %d, %d, '%s', '%s', '%s', '%s', '%s', '%s', '%s', '%s', '%s')",
        ID,
        rodovia,
        classeAcidente,
        nomeConcessionaria,
        data,
        tipoAcidente,
        meteoro,
        VISIB,
        DENOMINACAO,
        MUNICIPIO,
        REGIONAL_DER,
        JURISDICA0,
        LAT_FINAL,
        LON_FINAL
    );
}
```

Utilizamos as variáveis de instancia para criar o insert e adicionar essas variáveis em uma String.

Responsabilidade	Entrada	Saida
Usar as variáveis do objeto para criar um insert para a tabela Acidente.	X	String com o insert

!Atenção – Essa classe será muito utilizada como um objeto como vamos ver ela será uma lista das subclasses de Excel assim conseguiremos salvar cada acidente em um objeto com métodos para o insert

Classe RetornarFks:

A classe RetornarFks serve como apoio para RegistroAcidente, sempre que alguém for criar um objeto de RegistroAcidente, o construtor de RegistroAcidente chamará métodos de RetornarFks para retornar o número de iD do dado no Banco de Dados:

Por Exemplo:

- Envio o nome da empresa AutoBan para o construtor de RegistroAcidente
- O Construtor chama o método Fk_concessionária da classe RetornarFks
- E assim o método Fk_concessionária retorna o número de id da concessionária no Banco de Dados

Contando com os três métodos:

Método Fk_Concessionaria

```
public class RetonarFKs { 2 usages Gabriel-Akira-Endo +1

    public Integer Fk_Concessionaria(String nomeConcessionaria) { 1 usage Gabriel-Akira-Endo +1
        return switch (nomeConcessionaria.toUpperCase()) {
            case "AUTOBAN" -> 1;
            case "CART" -> 2;
            case "COLINAS" -> 3;
            case "ECOVIAS IMIGRANTES" -> 4;
            case "EIXOSP" -> 5;
            case "ENTREVIAS" -> 6;
            case "INTERVIAS" -> 7;
            case "LESTE PAULISTA" -> 8;
            case "NOROESTE PAULISTA" -> 9;
            case "NOVO LITORAL" -> 10;
            case "RAPOSO CASTELLO" -> 11;
            case "RENOVIAS" -> 12;
            case "RODOANEL" -> 13;
            case "ROTA" -> 14;
            case "SOROCABANA" -> 15;
            case "SPMAR" -> 16;
            case "SPVIAS" -> 17;
            case "TAMOIOS" -> 18;
            case "TEBE" -> 19;
            case "TIETÊ" -> 20;
            case "TRIANGULO" -> 21;
            case "VIAOESTE" -> 22;
            case "VIAPAULISTA" -> 23;
            case "VIARONDON" -> 24;
            default -> -1;
        };
    }
}
```

Retornando o Id da concessionária no Banco de Dados.

Responsabilidade	Entrada	Saida
Retonar o Id da concessionária de acordo com a String recebida	String nome da Concessionaria	Integer Do ID da concessionária

Método Fk_rodovia:

```
public Integer Fk_rodovia(String rodovia) { 1 usage Gabriel-Akira-Endo +1
    return switch (rodovia.toUpperCase()) {
        case "AVM030" -> 1;
        case "CTH231" -> 2;
        case "FORA" -> 3;
        case "NRC138" -> 4;
        case "PTM075" -> 5;
        case "SAS393" -> 6;
        case "SCD354" -> 7;
        case "NULL" -> 8;
        case "SP000" -> 9;
        case "SP019" -> 10;
        case "SP021" -> 11;
        case "SP029" -> 12;
        case "SP055" -> 13;
        case "SP063" -> 14;
        case "SP065" -> 15;
        case "SP070" -> 16;
        case "SP075" -> 17;
        case "SP079" -> 18;
        case "SP083" -> 19;
        case "SP088" -> 20;
        case "SP098" -> 21;
        case "SP099" -> 22;
        case "SP101" -> 23;
        case "SP113" -> 24;
        case "SP127" -> 25;
        case "SP147" -> 26;
        case "SP148" -> 27;
        case "SP150" -> 28;
        case "SP160" -> 29;
        case "SP191" -> 30;
        case "SP197" -> 31;
        case "SP209" -> 32;
        case "SP215" -> 33;
        case "SP225" -> 34;
    };
}
```

Retornando o Id da rodovia no Banco de Dados.

Responsabilidade	Entrada	Saida
Retonar o Id da rodovia de acordo com a String recebida	String Nome da Rodovia	Integer Do ID da rodovia

Método Fk_ClasseAcidente:

```
public Integer Fk_classeAcidente(String classeAcidente) { 1 usage  Gabriel-Akira-Endo +1
    return switch (classeAcidente.toUpperCase()) {
        case "ATROPELAMENTO DE ANIMAL" -> 1;
        case "ATROPELAMENTO DE PEDESTRE" -> 2;
        case "CAPOTAMENTO" -> 3;
        case "CHOQUE" -> 4;
        case "COLISÃO FRONTAL" -> 5;
        case "COLISÃO LATERAL" -> 6;
        case "COLISÃO TRANSVERSAL" -> 7;
        case "COLISÃO TRASEIRA" -> 8;
        case "ENGAVETAMENTO" -> 9;
        case "INCÊNDIO" -> 10;
        case "OBJETO CONTRA VEÍCULO" -> 11;
        case "OUTROS" -> 12;
        case "QUEDA" -> 13;
        case "SAÍDA DE PISTA" -> 14;
        case "SEM INFO/NULO/0" -> 15;
        case "TOMBAMENTO" -> 16;
        default -> -1; // Retorna null se não encontrar correspondência
    };
}
```

Retornando o Id do acidente no Banco de Dados.

Responsabilidade	Entrada	Saida
Retonar o Id da classe do acidente de acordo com a String recebida	String Nome da classe	Integer Do ID da classe

Classe ExcelModeloAtual:

A classe ExcelModeloAtual, é uma subclasse de Excel e com ela conseguimos tratar as planilhas mais atuais disponibilizadas.

De inicio já enviamos os dados base a superclasse usando o construtor, passando o arquivo a ser lido, as posições do Excel e o idPadrao :

```
public ExcelModeloAtual(String arquivoTratar,Integer idPadrao) { 1 usage  Lucas-TDO
    super(arquivoTratar,new Integer[]{1,2,5,6,7,8,10,11,13,14,15,16,17,18,20,21,22,23,25,26},idPadrao);
    this.registros = new ArrayList<>();
}
```

E declarando uma Lista de Objetos da classe RegistroAcidente, toda vez que uma linha do Excel for lida vai ser criado um novo objeto de RegistroAcidente com todos os dados salvos dentro do objeto.

```
public class ExcelModeloAtual extends Excel{ 2 usages lucas-tdo

    private List<RegistroAcidente> registros; 12 usages
```

Método tratarExcel:

Nisso chegamos ao método que trata o Excel em si, já tendo os dados do caminho do arquivo, id Padrão e as posições da célula vira uma questão de apenas passar pelo arquivo coletando todos os dados.

```
@Override @SuppressWarnings
public List<RegistroAcidente> tratarExcel() {
    Integer id = 1;

    try {
        Workbook workbook = new XSSFWorkbook(getArquivoTratar());
        Sheet sheet = workbook.getSheetAt(0);
        DataFormatter formatter = new DataFormatter();
        Integer posicoes[] = getPosicaoCélulasExcel();

        for (Row row : sheet) {
            if (row.getRowNum() == 0) continue;
            String registroArray[] = new String[posicoes.length];

            for (int i = 0; i < posicoes.length; i++) {
                Cell cell = row.getCell(posicoes[i]);
                registroArray[i] = cell != null ? formatter.formatCellValue(cell) : "";
            }

            if (registroArray[0].equals("SEM INFO/NULO/0")) {
                registros.add(new RegistroAcidente(Integer.parseInt(super.getIdPadrao()) + String.valueOf(id), registroArray[0], registroArray[1], registroArray[2], registroArray[3], registroArray[4], registroArray[5], registroArray[6], registroArray[7], registroArray[8], registroArray[9], registroArray[10], registroArray[11], registroArray[12], registroArray[13], registroArray[14], registroArray[15], registroArray[16], registroArray[17], registroArray[18], registroArray[19], registroArray[20], registroArray[21], registroArray[22], registroArray[23], registroArray[24], registroArray[25], registroArray[26], registroArray[27], registroArray[28], registroArray[29], registroArray[30], registroArray[31], registroArray[32], registroArray[33], registroArray[34], registroArray[35], registroArray[36], registroArray[37], registroArray[38], registroArray[39], registroArray[40], registroArray[41], registroArray[42], registroArray[43], registroArray[44], registroArray[45], registroArray[46], registroArray[47], registroArray[48], registroArray[49], registroArray[50], registroArray[51], registroArray[52], registroArray[53], registroArray[54], registroArray[55], registroArray[56], registroArray[57], registroArray[58], registroArray[59], registroArray[60], registroArray[61], registroArray[62], registroArray[63], registroArray[64], registroArray[65], registroArray[66], registroArray[67], registroArray[68], registroArray[69], registroArray[70], registroArray[71], registroArray[72], registroArray[73], registroArray[74], registroArray[75], registroArray[76], registroArray[77], registroArray[78], registroArray[79], registroArray[80], registroArray[81], registroArray[82], registroArray[83], registroArray[84], registroArray[85], registroArray[86], registroArray[87], registroArray[88], registroArray[89], registroArray[90], registroArray[91], registroArray[92], registroArray[93], registroArray[94], registroArray[95], registroArray[96], registroArray[97], registroArray[98], registroArray[99]));
            } else {
                registros.add(new RegistroAcidente(Integer.parseInt(super.getIdPadrao()) + String.valueOf(id), registroArray[0], registroArray[1], registroArray[2], registroArray[3], registroArray[4], registroArray[5], registroArray[6], registroArray[7], registroArray[8], registroArray[9], registroArray[10], registroArray[11], registroArray[12], registroArray[13], registroArray[14], registroArray[15], registroArray[16], registroArray[17], registroArray[18], registroArray[19], registroArray[20], registroArray[21], registroArray[22], registroArray[23], registroArray[24], registroArray[25], registroArray[26], registroArray[27], registroArray[28], registroArray[29], registroArray[30], registroArray[31], registroArray[32], registroArray[33], registroArray[34], registroArray[35], registroArray[36], registroArray[37], registroArray[38], registroArray[39], registroArray[40], registroArray[41], registroArray[42], registroArray[43], registroArray[44], registroArray[45], registroArray[46], registroArray[47], registroArray[48], registroArray[49], registroArray[50], registroArray[51], registroArray[52], registroArray[53], registroArray[54], registroArray[55], registroArray[56], registroArray[57], registroArray[58], registroArray[59], registroArray[60], registroArray[61], registroArray[62], registroArray[63], registroArray[64], registroArray[65], registroArray[66], registroArray[67], registroArray[68], registroArray[69], registroArray[70], registroArray[71], registroArray[72], registroArray[73], registroArray[74], registroArray[75], registroArray[76], registroArray[77], registroArray[78], registroArray[79], registroArray[80], registroArray[81], registroArray[82], registroArray[83], registroArray[84], registroArray[85], registroArray[86], registroArray[87], registroArray[88], registroArray[89], registroArray[90], registroArray[91], registroArray[92], registroArray[93], registroArray[94], registroArray[95], registroArray[96], registroArray[97], registroArray[98], registroArray[99]));
            }
            id++;
        }

        workbook.close();
    } catch (Exception e) {
        e.printStackTrace();
        e.getMessage();
    }

    System.out.println("Foram tratados " + registros.size() + " registros");
    return registros;
}
```

Informações importantes sobre o método:

- O método não recebe nada e retorna a lista declarada no início da classe citada anteriormente tratando os dados de acordo com a necessidade;
- O método salva todos os dados da linha no array de suporte "registroArray" e após isso cria um objeto de RegistroAcidente e adiciona a Lista registros;
- O método usa o getter da classe pai no array posicaoCélulasExcel para saber exatamente onde deve pegar os dados na planilha;
- O método ignora a primeira linha que é o cabeçalho;
- O método quando recebe null de uma célula transforma em uma string vazia;
- O método quando recebe hora como "SEM INFO/NULO/0" só adiciona a data a lista da classe RegistroAcidente

Após passar pelo método já temos uma lista cheia de objetos e cada objeto tem os dados do acidente contextualizado que podemos usar os métodos do objeto para fazer os inserts

Responsabilidade	Entrada	Saida
Ler as planilhas do excel mais recentes da artesp, trata os dados e manda para um objeto de RegistroAcidente	X	Lista de RegistroAcidente com os dados do acidente tratados

Método analisarErros:

Além disso temos o método AnalisarErros para checar se em algum objeto, alguma chave estrangeira não está fora do banco e assim retornando -1 .

```
public void analisarErros(){ no usages  lucas-tdo
    for (int i = 0; i < registros.size(); i++) {
        int fkConcessionaria = registros.get(i).getNomeConcessionaria();
        int fkRodovia = registros.get(i).getRodovia();
        int fkClasse = registros.get(i).getClasseAcidente();

        if(fkConcessionaria == -1 ){
            System.out.println("Registro com concessionaria inválida: " + registros.get(i).toString());
        }
        else if(fkRodovia == -1){
            System.out.println("Registro sem rodovia: " + registros.get(i).toString() );
        }
        else if(fkClasse == -1){
            System.out.println("Registro sem classe: " + registros.get(i).toString());
        }
    }
}
```

Responsabilidade	Entrada	Saida
Verificar possíveis erros ao pegar os ids de classe ,rodovia e concessionária	X	X

Classe ExcelModeloAntigo:

A classe ExcelModeloAntigo, é uma subclasse de Excel e com ela conseguimos tratar as planilhas mais antigas disponibilizadas.

!Atenção – Essa classe segue o mesmo padrão de ExcelModeloAtual se você já entendeu a classe anterior, não é necessário tentar entender a lógica dessa

De inicio já enviamos os dados base a superclasse usando o construtor, passando o arquivo a ser lido, as posições do Excel e o idPadrao :

```
public ExcelModeloAntigo(String arquivoTratar, Integer idPadrao) { 2 usages  lucas-tdo
    super(arquivoTratar, new Integer[]{0, 1, 4, 5, 6, 7, 8, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 24, 25}, idPadrao);
    this.registros = new ArrayList<>();
}
```

E declarando uma Lista de Objetos da classe RegistroAcidente, toda vez que uma linha do Excel for lida vai ser criado um novo objeto de RegistroAcidente com todos os dados salvos dentro do objeto.

```
public class ExcelModeloAntigo extends Excel { 4 usages  lucas-tdo
    private List<RegistroAcidente> registros; 11 usages
}
```

Método tratarExcel:

Nisso chegamos ao método que trata o Excel em si, já tendo os dados do caminho do arquivo, id Padrão e as posições da célula vira uma questão de apenas passar pelo arquivo coletando todos os dados.

```
@Override @SuppressWarnings
public List<RegistroAcidente> tratarExcel() {
    Integer id = 1;
    try {
        Workbook workbook = new HSSFWorkbook(getArquivoTratar());
        Sheet sheet = workbook.getSheetAt(0);
        DataFormatter formatter = new DataFormatter();
        Integer posicoes[] = getPosicaoCelulasExcel();

        for (Row row : sheet) {
            if (row.getRowNum() == 0) {
                continue;
            }
            String registroArray[] = new String[posicoes.length];
            for (int i = 0; i < posicoes.length; i++) {
                Cell cell = row.getCell(posicoes[i]);
                registroArray[i] = cell != null ? formatter.formatCellValue(cell) : "";
            }
            registros.add(new RegistroAcidente(Integer.parseInt(super.getIdPadrao()) + String.valueOf(id), registroArray[0], registroArray[1], registroArray[2], registroArray[3], registroArray[4], registroArray[5], registroArray[6], registroArray[7], registroArray[8], registroArray[9], registroArray[10], registroArray[11], registroArray[12], registroArray[13], registroArray[14], registroArray[15], registroArray[16], registroArray[17], registroArray[18], registroArray[19], registroArray[20], registroArray[21], registroArray[22], registroArray[23], registroArray[24], registroArray[25]));
            id++;
        }
        workbook.close();
    } catch (Exception e) {
        e.printStackTrace();
        e.getMessage();
    }

    System.out.println("Foram tratados " + registros.size() + " registros");
    return registros;
}
```

Informações importantes sobre método:

- O método não recebe nada e retorna a lista citada anteriormente tratando os dados de acordo com a necessidade;
- O método salva todos os dados da linha no array de suporte "registroArray";
- O método usa o getter da classe pai no array posicaoCelulasExcel para saber exatamente onde deve pegar os dados na planilha;
- O método ignora a primeira linha que é o cabeçalho
- O método quando recebe null de uma célula transforma em uma string vazia
- Diferente de tratarExcel na classe ExcelModeloAtual os excel antigos a data e hora estão juntas assim são enviados direto diminuindo o array de suporte registroArray

Após passar pelo método já temos uma lista cheia de objetos e cada objeto tem os dados do acidente contextualizado que podemos usar os métodos do objeto para fazer os inserts

Responsabilidade	Entrada	Saida
Ler as planilhas do excel com modelo mais antigo da artesp, trata os dados e manda para um objeto de RegistroAcidente	X	Lista de RegistroAcidente com os dados do acidente tratados

Método analisarErros:

Além disso temos o método AnalisarErros para checar se em algum objeto, alguma chave estrangeira não está fora do banco e assim retornando -1

```

public void analisarErros(){ no usages @lucas-tdo
    for (int i = 0; i < registros.size(); i++) {
        int fkConcessionaria = registros.get(i).getNomeConcessionaria();
        int fkRodovia = registros.get(i).getRodovia();
        int fkClasse = registros.get(i).getClasseAcidente();

        if(fkConcessionaria == -1){
            System.out.println("Registro com concessionaria inválida: " + registros.get(i).toString());
        }
        else if(fkRodovia == -1){
            System.out.println("Registro sem rodovia: " + registros.get(i).toString() );
        }
        else if(fkClasse == -1){
            System.out.println("Registro sem classe: " + registros.get(i).toString());
        }
    }
}

```

Responsabilidade	Entrada	Saida
Verificar possíveis erros ao pegar os ids de classe ,rodovia e concessionária	X	X

Classe ConexaoBanco:

Essa é uma classe onde fazemos toda a conexão ao banco de dados mySql

```
package school.sptech;

import org.apache.commons.dbcp2.BasicDataSource;
import org.springframework.jdbc.core.JdbcTemplate;

public class ConexaoBanco { 2 usages  @ lucas-tdo
    private final JdbcTemplate jdbcTemplate; 2 usages
    private final BasicDataSource basicDataSource; 2 usages

    public ConexaoBanco(){ 1 usage  @ lucas-tdo
        BasicDataSource basicDataSource = new BasicDataSource();
        basicDataSource.setUrl("jdbc:mysql://192.168.0.22:3306/waysafe");
        basicDataSource.setUsername("Way_Safe");
        basicDataSource.setPassword("Urubu100");

        this.basicDataSource = basicDataSource;
        this.jdbcTemplate = new JdbcTemplate(basicDataSource);
    }

    public BasicDataSource getBasicDataSource() { no usages  @ lucas-tdo
        return basicDataSource;
    }

    public JdbcTemplate getJdbcTemplate() { 1 usage  @ lucas-tdo
        return jdbcTemplate;
    }
}
```

Precisando apenas criar o objeto BasicDataSource

Adicionando o Url do banco:

```
basicDataSource.setUrl("jdbc:mysql://<IPV4 da
máquina>:<Porta>/<BancoDeDados>");
```

Adicionando o usuário da máquina

```
basicDataSource.setUsername("<Nome Do User>");
```

E senha do usuário:

```
basicDataSource.setPassword("<Senha do Usuário>");
```

A classe tem sim outras coisas como getter e setter mas afim de aprendizado sobre o código não são tão importantes.

Classe ComandosJavaBanco:

Essa é a classe onde centralizamos os objetos com os dados e a conexão do Banco assim conseguindo adicionar finalmente os dados no MySql

Método save:

Recebe duas strings para fazer o insert na tabela Acidentes e Vitimas

```
public void save(String insertAcidente,String insertVitimas) { 1 usage  @lucas-tdo
    jdbcTemplate.update("INSERT INTO ACIDENTE (idACIDENTE,fk_rodovias, fk_classe_acid, fk_empresa, data_hora, tipo_acidente, metereologia, visibilidade, denominacao, municipio, regional_der, jurisdicao, latitude,
    jdbcTemplate.update("INSERT INTO VITIMAS (fk_acidente, vitima_ilesa, vitima_fatal, vitima_fer_leve, vitima_fer_media, vitima_fer_grave , vitimas_fer_seminfo) VALUES ".concat(insertVitimas));
}
```

Responsabilidade	Entrada	Saida
Executar os comandos SQL para inserir em ACIDENTE E VITIMAS	String Com os inserts de Acidente String Com os inserts de Vitimas	X

Método saveLog:

Recebe três strings para fazer o insert na tabela Log sabendo erros em arquivos específicos.

```
public void saveLog(String status , String mensagem,String arquivo){ 1 usage  @lucas-tdo
    jdbcTemplate.update("insert into LOG(status,mensagem, arquivo) values (?,?,?)",status,mensagem,arquivo);
}
```

Responsabilidade	Entrada	Saida
Registrar no Banco o Log da execução	String Com os inserts do Log	X

Método saveLote:

O método recebe a Lista de RegistroAcidente e o nome do arquivo que está sendo tratado.

```
public void saveLote(List<RegistroAcidente> registros , String arquivoTratar){ 3 usages  @lucas-tdo
```

Então passa por cada objeto da lista recebida e chama os métodos RetornarAcidente e RetornarVitimas da Classe de RegistroAcidente e recebe os inserts desses métodos os acumulando em variáveis de apoio **Acidentes e Vitimas**.

Após esse processo ser realizado mil vezes ou chegar no último objeto, chama o método `save` para salvar no banco todos os inserts armazenados em `Acidentes e Vitimas`.

```
if(i==registros.size()-1 || qtdRegistros==1000){
    LocalDateTime dataHora = LocalDateTime.now();
    DateTimeFormatter dataFormato = DateTimeFormatter.ofPattern(" dd/MM/yyyy '('EEEE')' hh:mm:ss a");
    String dataFormatada = dataHora.format(dataFormato);
    String mensagem = "Registro de "+ (ultimaAdicionada)+ " a "+ (i+1) + " adicionados ao banco de dados, do arquivo "+arquivoTratar;
    System.out.println(dataFormatada+" "+mensagem);
    save(acidentes,vitimas);
    saveLog( status: "Sucesso",mensagem,arquivoTratar);
    qtdRegistros=0;
    ultimaAdicionada=i;
    acidentes="";
    vitimas="";
}
```

Além disso quando se iniciar o processo de salvar no Banco após os 1000 inserts se cria um Log:

```
LocalDateTime dataHora = LocalDateTime.now();
DateTimeFormatter dataFormato = DateTimeFormatter.ofPattern(" dd/MM/yyyy '('EEEE')' hh:mm:ss a");
String dataFormatada = dataHora.format(dataFormato);
```

Com duas intenções apresentar no cmd do usuário e salvar no banco de dados chamando o método já explicado `saveLote` e assim salvando toda a execução.

Responsabilidade	Entrada	Saida
Salvar os registros da Lista recebida em lotes de 1000 em 1000 e registrar os logs da execução	Lista Com todos os objetos com os dados String Com o nome do arquivo	X

Classe Main:

Essa classe é onde apenas chamamos nossa classe para realizar as funções

```
public class Main { @lucas-tdo
    public static void main(String[] args) { @lucas-tdo
        ConexaoBanco cnx = new ConexaoBanco();
        ComandosJavaBanco comandos = new ComandosJavaBanco(cnx.getJdbcTemplate());

        ExcelModeloAtual excel25 = new ExcelModeloAtual( arquivoTratar: "acidentes_2025.xlsx", idPadrao: 2025);
        ExcelModeloAntigo excel24 = new ExcelModeloAntigo( arquivoTratar: "acidentes_2024.xlsx", idPadrao: 2024);
        ExcelModeloAntigo excel23 = new ExcelModeloAntigo( arquivoTratar: "acidentes_2023.xlsx", idPadrao: 2023);
        List<RegistroAcidente> lote25 = excel25.tratarExcel();
        List<RegistroAcidente> lote24 = excel24.tratarExcel();
        List<RegistroAcidente> lote23 = excel23.tratarExcel();
        comandos.saveLote(lote25,excel25.getArquivoTratar());
        comandos.saveLote(lote24,excel24.getArquivoTratar());
        comandos.saveLote(lote23,excel23.getArquivoTratar());
    }
}
```

Criamos a conexão com o Banco e adicionamos no construtor da ComandosJavaBanco

Após isso chamamos a classe CORRETA para o arquivo em si em 2025 chamo a classe ExcelModeloAtual e mando o caminho do arquivo e o Id padrão.

E faço o mesmo para 24 e 23 usando o ExcelModeloAntigo

```
ExcelModeloAtual excel25 = new ExcelModeloAtual( arquivoTratar: "acidentes_2025.xlsx", idPadrao: 2025);
ExcelModeloAntigo excel24 = new ExcelModeloAntigo( arquivoTratar: "acidentes_2024.xlsx", idPadrao: 2024);
ExcelModeloAntigo excel23 = new ExcelModeloAntigo( arquivoTratar: "acidentes_2023.xlsx", idPadrao: 2023);
```

Crio três listas para armazenar os objetos da planilha chamando tratarExcel em cada um dos objetos criados anteriormente

```
List<RegistroAcidente> lote25 = excel25.tratarExcel();
List<RegistroAcidente> lote24 = excel24.tratarExcel();
List<RegistroAcidente> lote23 = excel23.tratarExcel();
```

E por fim chamo os comandos SaveLote para salvar todos os dados que foram para essas listas

```
comandos.saveLote(lote25,excel25.getArquivoTratar());
comandos.saveLote(lote24,excel24.getArquivoTratar());
comandos.saveLote(lote23,excel23.getArquivoTratar());
```

E aí sim os dados estarão salvos

idACIDENTE	R_rodovias	R_classe_acd	R_empresa	data_hora	tpo_acidente	metereologia	viabilidade	denominacao	municipio	regional_der	jurisdcao	latitude	longitude
202388	51	1	21	2023-01-02 00:11:33	ATROP. ANIMAL	BOA	BOA	WASHINGTON LUI	PINDORAMA	SÃO JOSÉ DO RI...	ESTADUAL	-2.120.186.858.493...	-4.893.867.853.329...
202389	58	1	1	2023-01-02 00:27:28	ATROP. ANIMAL	BOA	BOA	ANHANGUERA	SÃO PAULO	SÃO PAULO	ESTADUAL	-2.340.593.239	-4.682.719.158
202390	23	6	39	2023-01-02 00:41:36	COLISÃO LATERAL	BOA	PARCIAL	JORNALISTA FRANCISCO AGUIRRE PROENÇA	CAMPINAS	RIO CLARO	ESTADUAL	-230.206.483.885.559	-475.010.917.722.248
202391	48	1	24	2023-01-02 01:16:19	ATROP. ANIMAL	BOA	PARCIAL	MARECHAL RONDON	COROAPOS	ARACATUBA	ESTADUAL	-2.138.386	-5.025.411
202392	58	1	1	2023-01-02 01:36:05	ATROP. ANIMAL	BOA	BOA	ANHANGUERA	JUNDIAÍ	CAMPINAS	ESTADUAL	-23.230.639	-4.687.295
202393	42	8	2	2023-01-02 04:19:16	COLISÃO TRASERA	BOA	BOA	RAPOSO TAVARES	PRESIDENTE EPITÁCIO	PRESIDENTE PR...	ESTADUAL	-217.937.386.834.717	-521.249.351.501.465
202394	30	1	7	2023-01-02 05:02:54	ATROP. ANIMAL	BOA	BOA	VILSON FARIAS	AGUAS	RIO CLARO	ESTADUAL	-2.237.817.999	-4.740.841.997
202395	43	4	22	2023-01-02 06:01:58	CHOXO F-OFENSA	BOA	BOA	CASTRILHO BRANCO	BARBOSA	SÃO PAULO	ESTADUAL	-2.351.061.099	-4.681.563.556

Espaço para possíveis melhorias Seção Bucket e atualizar Banco